Linux网络--Http浏览器与连接演进

- 1. 浏览器为什么不是只请求一个文件
  - 1.1 浏览器如何解释响应
  - 1.2 缓存的角色
- 2. HTTP/1.0 的短连接
  - 2.1 连接关闭也能表示正文结束
- 3. HTTP/1.1 的持久连接
  - 3.1 管线化与队头阻塞
  - 3.2 浏览器连接池
- 4. 长连接故障为何会导致图片不出现
  - 4.1 响应边界错误
  - 4.2 服务端提前关闭
  - 4.3 请求没有读完整
    - 4.3.1 读取完整请求
    - 4.3.2 发送完整响应
    - 4.3.3 保证二进制正文完整
- 5. HTTP/2 与 HTTP/3
  - 5.1 HTTP/2 多路复用
  - 5.2 HTTP/3 与 QUIC
- 6. 浏览器兼容性与协议容错
- 7. 实验：比较短连接和持久连接
  - 7.1 短连接测试
  - 7.2 持久连接测试
  - 7.3 观察结果

1. 浏览器为什么不是只请求一个文件

浏览器首先请求 HTML，然后解析 HTML 中的资源引用，继续请求 CSS、JavaScript、图片、字体、音视频和 favicon：

```
<!doctype html>
<html>
  <head>
    <link rel="stylesheet" href="/css/app.css">
    <script src="/js/app.js" defer></script>
  </head>
  <body>
    <img src="/image/cover.png" alt="封面">
```

```
    <video src="/video/demo.mp4" controls></video>
  </body>
</html>
```

一个包含十张不同图片的页面，除 HTML 外至少还要发起十次图片请求。同一资源被重复引用时，浏览器通常只请求一次并利用缓存渲染多次。

浏览器会根据资源依赖、优先级、缓存和并发限制安排请求顺序。页面“看起来是一页”，在网络层却是一组请求集合。

1.1 浏览器如何解释响应

服务器必须通过 `Content-Type` 告知正文类型：

```
Content-Type: text/html; charset=utf-8
Content-Type: text/css; charset=utf-8
Content-Type: text/javascript; charset=utf-8
Content-Type: image/png
Content-Type: video/mp4
```

浏览器拿到 HTML 后构建 DOM，再解析 CSS 和脚本。图片、视频等资源加载失败时，页面主文档可能仍然显示，但局部内容会缺失。

1.2 缓存的角色

浏览器缓存可以减少重复请求。常见策略包括：

- `Cache-Control` 指定缓存时长和重新验证规则。
- `ETag` 表示资源版本。
- `Last-Modified` 表示最后修改时间。
- `Expires` 指定绝对过期时间。

资源未变化时，服务器可以返回 304，不重复发送正文。

2. HTTP/1.0 的短连接

HTTP/1.0 默认非持久连接：

```
建立 TCP -> 请求 HTML -> 响应 -> 关闭
建立 TCP -> 请求 CSS  -> 响应 -> 关闭
建立 TCP -> 请求图片 -> 响应 -> 关闭
```

每个资源都要重新建立连接，会增加握手和慢启动成本。早期网站资源较少，这种方式还可以接受；现代页面通常包含大量图片、脚本和样式。

HTTP/1.0 可以在请求或响应中显式使用：

```
Connection: keep-alive
```

表示双方希望复用连接，但它不是 HTTP/1.0 的默认行为。

2.1 连接关闭也能表示正文结束

HTTP/1.0 响应如果既没有 `Content-Length`，也没有分块传输，客户端可以读取到连接关闭为止。这种方式要求服务器明确关闭连接，无法继续复用。

现代服务端应尽量使用长度或分块规则明确报文边界，降低浏览器误判和代理兼容问题。

3. HTTP/1.1 的持久连接

HTTP/1.1 默认使用持久连接。一个连接上可以依次发送多个请求，每个响应按顺序返回。

```
TCP 建立
  -> 请求 HTML -> 响应 HTML
  -> 请求 CSS  -> 响应 CSS
  -> 请求图片 -> 响应图片
  -> 连接空闲或关闭
```

服务端不支持复用时，应明确返回：

```
Connection: close
```

并在完整发送响应后关闭连接。若服务端声明 HTTP/1.1 但仍按短连接处理，客户端可能尝试复用已被关闭的连接，导致第二个请求写入失败。

3.1 管线化与队头阻塞

HTTP/1.1 允许在收到前一个响应前发送后续请求，称为管线化。但响应必须严格按请求顺序返回：

```
请求 A -> 请求 B -> 请求 C
响应 A -> 响应 B -> 响应 C
```

如果响应 A 很慢，后面的响应不能越过它，形成队头阻塞。多数现代浏览器不会依赖 HTTP/1.1 管线化，而会建立多个并行连接或使用 HTTP/2。

3.2 浏览器连接池

浏览器通常会为同一站点维护多条连接，并行获取页面资源。长连接减少重复建连，但不等于整个页面只使用一条连接。

“一个网页几十个资源”和“一个连接多个请求”是两个不同问题：

- 资源数量决定页面需要多少次请求。
- 连接模型决定这些请求如何分配到多个连接和多次请求响应中。

4. 长连接故障为何会导致图片不出现

如果出现首页正常、图片不显示的故障。常见原因不是单一 Header 错误，而是请求读取、响应边界和连接生命周期不一致。

4.1 响应边界错误

如果服务器返回 HTTP/1.1，但没有完整的 `Content-Length`，又没有使用合法的结束规则，浏览器就不知道第一个响应何时结束，可能继续等待，因而不再发起图片请求。

4.2 服务端提前关闭

浏览器收到首页响应后尝试复用连接发送图片请求，但服务端已经在发送首页后关闭 Socket，第二次写入就会失败。

正确选择有两种：

1. 服务端真正支持持久连接，循环读取下一条请求。
2. 服务端只支持短连接，返回 `Connection: close` 并关闭连接。

4.3 请求没有读完整

服务器只调用一次 `read()`，可能只读到 Header 的一部分，或者没有读完正文。它提前构造响应后，剩余请求数据还会留在连接中，影响后续请求。

4.3.1 读取完整请求

```
读取到 Header 结束的空行
  -> 解析 Content-Length 或 Transfer-Encoding
  -> 继续读取正文直到完整
  -> 处理请求
```

4.3.2 发送完整响应

`write()` 可能只写入部分数据，循环直到全部响应字节发送完成。

4.3.3 保证二进制正文完整

图片和视频必须按二进制读取，不能按行读取。正文长度应是实际字节数。

5. HTTP/2 与 HTTP/3

5.1 HTTP/2 多路复用

HTTP/2 使用二进制帧，在一条 TCP 连接上并发传输多个流：

```
一个 TCP 连接
  -> Stream 1: HTML
  -> Stream 3: CSS
  -> Stream 5: Image
```

多个请求的响应可以交错传输，不再要求整个 HTTP 消息按顺序独占连接。HPACK 还能压缩重复 Header。

HTTP/2 降低了 HTTP 层的队头阻塞，但底层 TCP 丢失一个段时，所有流仍可能等待重传。

HTTP/2 服务端推送曾经被提出用于提前发送资源，但主流浏览器已停止或限制使用，不应作为必要设计。

5.2 HTTP/3 与 QUIC

HTTP/3 使用基于 UDP 的 QUIC：

- TLS 1.3 集成在连接建立过程中。
- 多个独立流并行传输。
- 单个流丢包不必阻塞其他流。
- 支持连接迁移等现代网络能力。

UDP 本身不保证可靠性，QUIC 在用户态实现可靠传输、拥塞控制和加密。

6. 浏览器兼容性与协议容错

不同浏览器内核和版本可能对 HTML、CSS、JavaScript 和 HTTP 边界处理存在差异。浏览器厂商会遵守标准，但容错能力并不完全相同。

开发时不能依赖以下行为：

- 缺少 `Content-Length` 也一定能正确判断正文结束。
- 任意错误 Header 都会被所有浏览器自动修复。
- 不规范的 `Content-Type` 一定不影响脚本执行。
- 所有浏览器都会用相同方式处理长连接复用。

应通过浏览器开发者工具查看 Network、Response Headers、Content-Type、Length、Connection 和 Timing，并通过不同浏览器或 curl 对照验证。

7. 实验：比较短连接和持久连接

7.1 短连接测试

让服务器返回：

```
HTTP/1.0 200 OK
Content-Type: text/html
Content-Length: 13
Connection: close

hello, world!
```

浏览器会把每个资源作为独立连接处理，服务器也可以在每次响应后安全关闭 Socket。

7.2 持久连接测试

让服务器返回：

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 13
Connection: keep-alive

hello, world!
```

服务端必须在发送后继续读取下一条请求，并正确解析新请求边界。不能发完第一条响应就直接退出连接处理函数。

7.3 观察结果

可使用：

```
curl -i http://127.0.0.1:8080/
curl -v http://127.0.0.1:8080/
```

浏览器开发者工具中重点观察：

- 首页和图片是否分别出现请求。
- 多个请求是否复用同一连接。
- 响应正文是否完整。
- 是否有请求一直处于 pending。
- 服务端日志中的进程或连接编号是否变化。

下一篇进入用户状态管理：Linux网络--HttpCookie与Session